

Modeling and Optimizing a Battery Charging Profile

Team 3:

- Cris Cervantes Gonzalez
- Soliana Hadera
- Vannara Net
- Aren Safarian

Table of Contents

Modeling and Optimizing a Battery Charging Profile.....	1
Break Down the Problem.....	1
Task 1: Fit the voltage equation to a lithium-ion battery profile.....	2
Task 2: Plot the electrical terms that express how energy moves in a circuit.....	10
Task 3: Compute analytical results.....	12
Interpretation of Results.....	15
Optional Extension (intermediate):.....	16
Identifying Charging Phases.....	16
Analyzing Each Phase Separately.....	18
Comparing Power and Efficiency of Phases.....	21
Modeling and Analyzing Different Charging Profiles Using Simulink.....	24

Break Down the Problem

In the space below, define the problem statement:

- This project utilizes MATLAB to analyze lithium-ion battery charging using a specified dataset. This involves fitting the given voltage equation to the battery profile, modeling relevant quantities such as voltage, current, and power to express the movement of energy through a circuit, and computing analytical results. By the end of this project, we will have estimated τ , determined the necessary time for the battery to reach 80% and 100% charge, and computed results regarding energy and power, including resistive energy loss.

List the requirements, constraints, and success criteria for the project:

- The requirements of this project are importing the battery dataset, fitting the voltage equation to the charging model, plotting voltage vs. time, current vs. time, and power vs. time graphs, estimating τ , calculating rate of voltage change over specific time intervals, computing time required to reach 80% and 100% charge, using integration to calculate total energy delivered to the battery, estimating power and energy loss, and summarizing results using a table.
- This project is constrained by the provided battery dataset and software limitations. All requirements must be fulfilled using the MATLAB live script, and all data must be derived from the imported dataset. Additionally, several computations in this code hinge upon estimations rather than strictly formulaic calculations. The

voltage equation must also be assumed to be an appropriate and accurate approximation of the lithium-ion battery profile. We must also work collaboratively and make time to meet frequently despite busy schedules.

- Success criteria for this project entail completing all the requirements while adhering to the above constraints. All code must run without error and effectively answer all questions, and team members must be able to pinpoint and thoroughly explain their contribution to the project.

What is your proposed solution?

- Our proposed solution is to complete tasks individually using the MATLAB live script template and collaborate to ensure a quality, complete project. We will complete tasks in order and maintain regular communication.

List the steps you will need to take to achieve your proposed solution:

- Complete teamwork agreement and establish team responsibilities
- Complete MATLAB summer bridge course to build familiarity with MATLAB software
- Maintain communication through Discord and meet regularly to keep up with deadlines
- Complete tasks sequentially and collaboratively conduct quality assurance
- Review completed project, make necessary edits, and proofread all material
- Compile data in a final presentation or live script

What challenges do you face in the process of achieving the solution?

- We face challenges with using MATLAB software to complete tasks, as this is a relatively new skill for most of us. Certain tasks, such as producing goodness-of-fit statistics and utilizing calculus functions, may prove to be difficult. Additionally, we must manage our own schedules to meet with one another and ensure tasks are punctually completed. Time must be well-managed and due dates must be met. Code must be thoroughly proofread and edited, which requires strict attention to detail and team coordination.

Task 1: Fit the voltage equation to a lithium-ion battery profile

Given real time-series voltage data for a lithium-ion battery, use MATLAB Curve Fitting Toolbox to fit the

equation $V(t) = V_{\max} \left(1 - e^{-\frac{t}{RC}} \right)$ and model the battery charging profile as a simple RC-circuit analog.

Load MathWorks example lithium-ion batteries data by following the steps [here](#). The page also includes information about the dataset, how the data was recorded, and the battery's maximum voltage.

Note: The battery from this example dataset has been cycled to failure (charged and discharged repeatedly). For this project, you may choose to use data from a single cycle and focus only on the period of time within a cycle that the battery is being charged. The battery is fully charged at 3.6V. This example battery uses a dynamic fast charging profile, but for this project, you will attempt to model the charging profile using a simple exponential capacitor charging equation.

Then, follow the workflow outlined in the [documentation](#) to fit the voltage equation to your selected data. Plot the data and fitted curve in the same figure and, finally, print the goodness-of-fit statistics.

```
% STEP 1: Load the example battery dataset
```

```
dataFile
= matlab.internal.examples.downloadSupportFile("predmaint","batteryagingdata/
singlecell/v1/singleCellLifeTimeData.zip"); % Downloads data file as
instructed
unzip(dataFile); % Unzips data file for use
load("singleCellLifeTimeData.mat"); % Loads data file
head(data,5); % Displays 10,000 rows of the dataset
```

Data_Point	Test_Time	DateTime	Step_Time	Step_Index	Cycle_Index	Current
1	0	13-May-2017 03:21:40	-1780	0	0	0
2	0.0001	13-May-2017 03:21:40	-1780	0	0	0
3	9.9983	13-May-2017 03:21:50	-1770	0	0	0
4	20.002	13-May-2017 03:22:00	-1760	0	0	0
5	30.001	13-May-2017 03:22:10	-1750	0	0	0

```
% STEP 2: Pre-process the data to focus on a single cycle, and the period of
time
```

```
% within that cycle that the battery was being charged
```

```
cycleNumber = input("Enter cycle number (1-876): "); % Prompts user for
cycle number (1-876)
```

```
cycleData = data(data.Cycle_Index == cycleNumber, :); % Focuses data on the
cycle selected
```

```
cycleData = cycleData(cycleData.Current > 0, :); % Focuses data on the
period of charging, as indicated by positive current
```

```
cycleData.DateTime = seconds(cycleData.DateTime - cycleData.DateTime(1)); %
Initializes time at 0 seconds
```

```
figure; % Opens figure
```

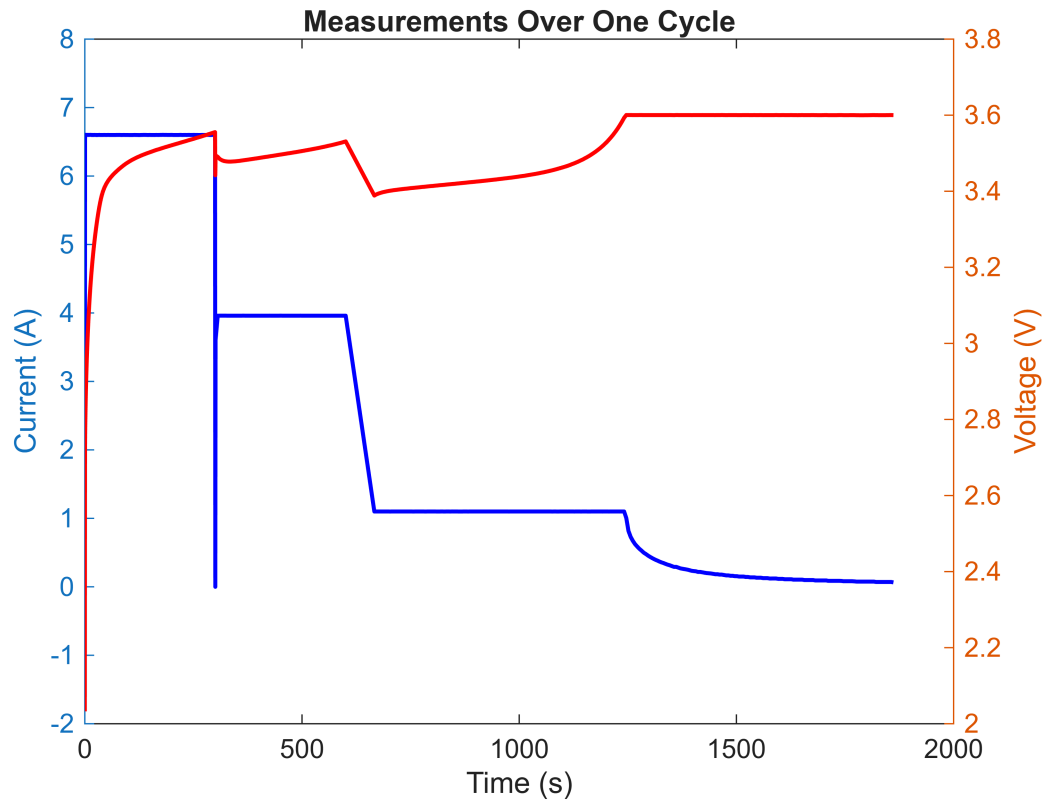
```
yyaxis left; % Selects left y-axis
```

```
plot(cycleData.DateTime,cycleData.Current, '-b', 'LineWidth', 1.5); % Plots
time on x-axis and current on left y-axis with a blue line and 1.5 line width
ylabel('Current (A)'); % Labels left y-axis
ylim([-2 8]) % Sets y-axis bounds from -2 to 8
```

```
yyaxis right; % Selects right y-axis
```

```
plot(cycleData.DateTime,cycleData.Voltage, '-r', 'LineWidth', 1.5); % Plots
time on x-axis and current on right y-axis with a red line and 1.5 line width
ylabel('Voltage (V)'); % Labels right y-axis
xlabel("Time (s)"); % Labels x-axis
```

```
title('Measurements Over One Cycle'); % Titles graph
```



```
bParser = batteryTestDataParser(data); % Creates parser object "bparser"
```

```
% Specifies properties corresponding to variable names
```

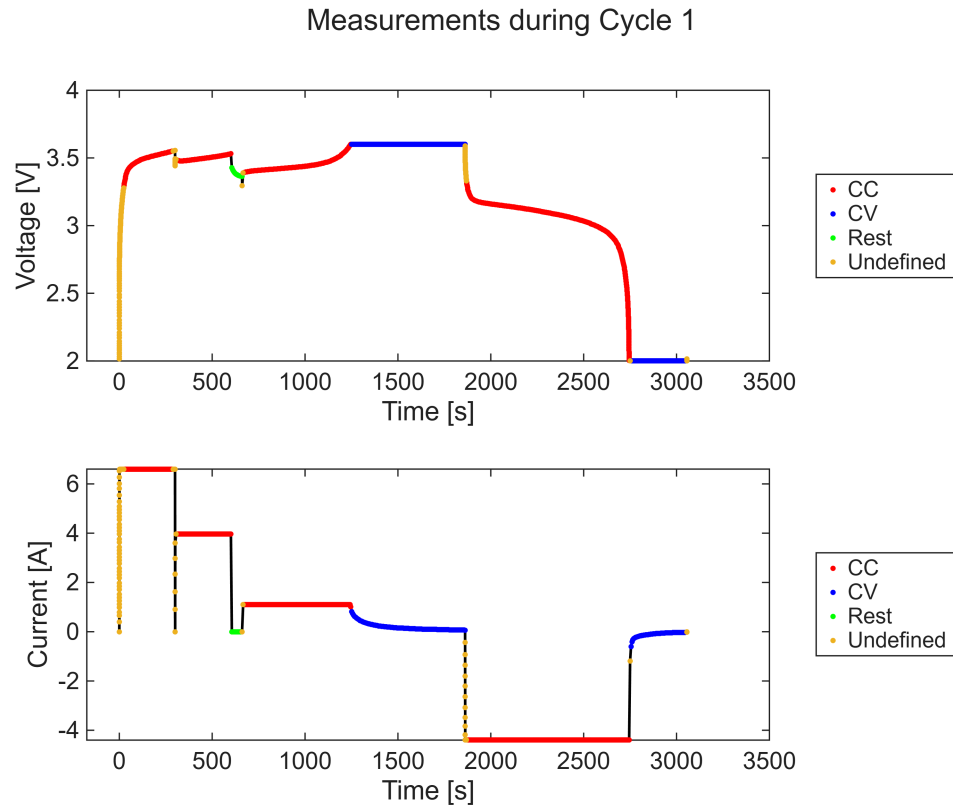
```
bParser.CurrentVariable = 'Current';
bParser.VoltageVariable = "Voltage";
bParser.TimeVariable = "DateTime";
bParser.CycleIndexVariable = 'Cycle_Index';
bParser.StepIndexVariable = 'Step_Index';
```

```
segmentedRawDataTable = segmentData(bParser); % Uses segmentData to identify
CyclingModes and CyclingPhases, and flags invalid data points
head(segmentedRawDataTable,5) % Displays the 5 rows of the segmented data
```

Data_Point	Test_Time	DateTime	Step_Time	Step_Index	Cycle_Index	Current
1	0	13-May-2017 03:21:40	-1780	0	0	0
2	0.0001	13-May-2017 03:21:40	-1780	0	0	0
3	9.9983	13-May-2017 03:21:50	-1770	0	0	0
4	20.002	13-May-2017 03:22:00	-1760	0	0	0
5	30.001	13-May-2017 03:22:10	-1750	0	0	0

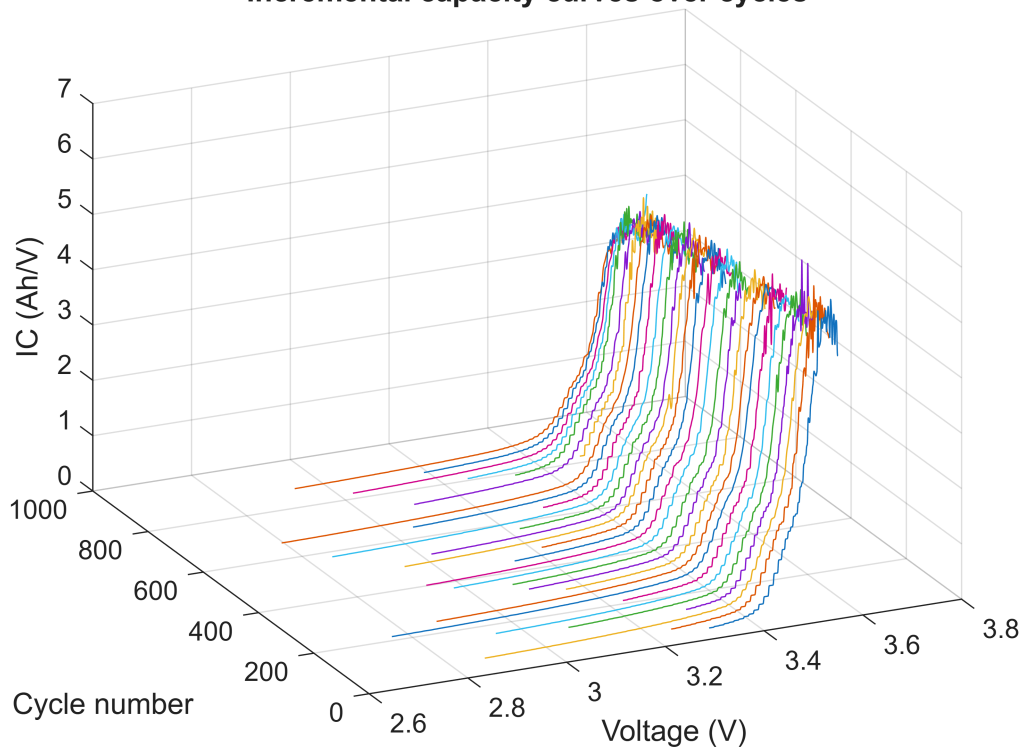
```
cycleList =
unique(segmentedRawDataTable(segmentedRawDataTable.IsValid, :).Cycle_Index);
% Creates cycleList, a list of unique cycle numbers from data points
```

```
hPlotRawMeasurementsWithSegments(segmentedRawDataTable, bParser,
cycleNumber) % Makes a plot of data and highlights different segments
```



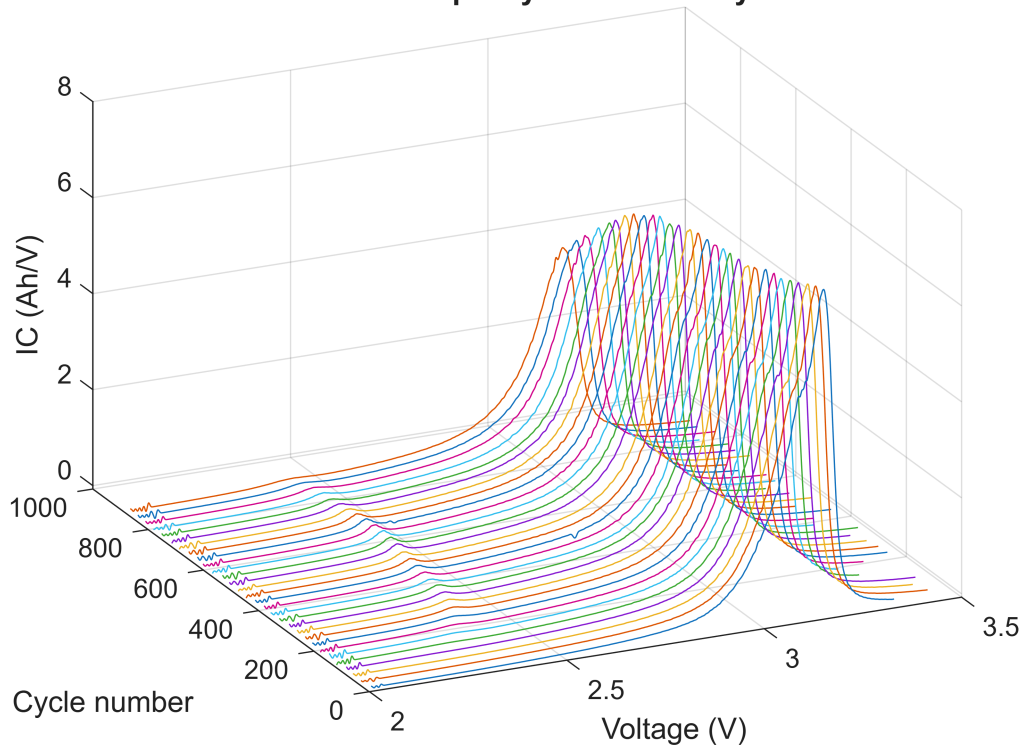
```
differentialTable = computeDifferentialCurves(bParser); % Creates
differential table by computing differential curves of data points
cycleList = sort(unique(differentialTable.(bParser.CycleIndexVariable))); %
Creates unique sorted list of cycle numbers
cyclingPhase = "Charge"; % Selects the charging phase of the cycle
stepIndex = 5; % Selects step 5
plotInterval = 30; % Plots cycles in intervals of 30
curveType = 'IC'; % Selects "incremental capacity" for the curve type
hPlotCurvesforCycles(differentialTable, bParser, cycleList, cyclingPhase,
stepIndex, plotInterval, curveType) % Plots the differential curves with
respect to the named parameters
```

Incremental capacity curves over cycles



```
differentialTable = computeDifferentialCurves(bParser); % Creates
differential table by computing differential curves of data points
cycleList = sort(unique(differentialTable.(bParser.CycleIndexVariable))); %
Creates unique sorted list of cycle numbers
cyclingPhase = "Discharge"; % Selects the discharging phase of the cycle
stepIndex = 10; % Selects step 10
plotInterval = 30; % Plots cycles in intervals of 30
curveType = 'IC'; % Selects "incremental capacity" for the curve type
hPlotCurvesforCycles(differentialTable, bParser, cycleList, cyclingPhase,
stepIndex, plotInterval, curveType) % Plots the differential curves with
respect to the named parameters
```

Incremental capacity curves over cycles



% STEP 3: Follow the provided workflow to fit the voltage equation to your selected

% data. Hint: V_{max} is known, so the only unknown in the equation is RC (also known as τ , the RC time constant - it is equivalent to the time it takes for a capacitor to reach ~63.2% of its max voltage). When you are fitting the curve to the data, you are essentially estimating the unknown parameter, τ .

% Assigns variables to the appropriate columns of the dataset

```
voltage = cycleData.Voltage;
time = cycleData.DateTime;
```

```
Vmax = max(cycleData.Voltage); % Finds the maximum voltage ( $V_{max}$ )
```

```
VTau = 0.632 * Vmax; % Estimates the voltage corresponding to the tau value as specified above
```

```
indexTau = find(voltage >= VTau, 1); % Finds the index number of voltages that reach VTau and selects the first
```

```
starting = time(indexTau); % Finds the corresponding time and selects it as the starting point in the fitted curve
```

```
fo = fitoptions('Method', 'NonlinearLeastSquares', ...
```

```

        'StartPoint', starting); % Selects fit options with a nonlinear
least squares regression curve and begins at the starting point

ft = fittype('Vmax * (1 - exp(-time/tau))', ...
    'independent', 'time', ...
    'coefficients', 'tau', ...
    'problem', 'Vmax', ...
    'options', fo); % Defines the voltage equation for the curve fit, time
as the independent variable,
                        % tau as an unknown coefficient, Vmax as a known entity,
and uses the fit options above

[f, gof] = fit(time, voltage, ft, 'problem', Vmax); % Returns the fitted
curve and goodness-of-fit statistics according to the named parameters

fittedCurve = f(time); % Applies the fitted curve to each time value

% STEP 4: Plot the data and the fitted curve in the same figure.

figure; % Opens figure

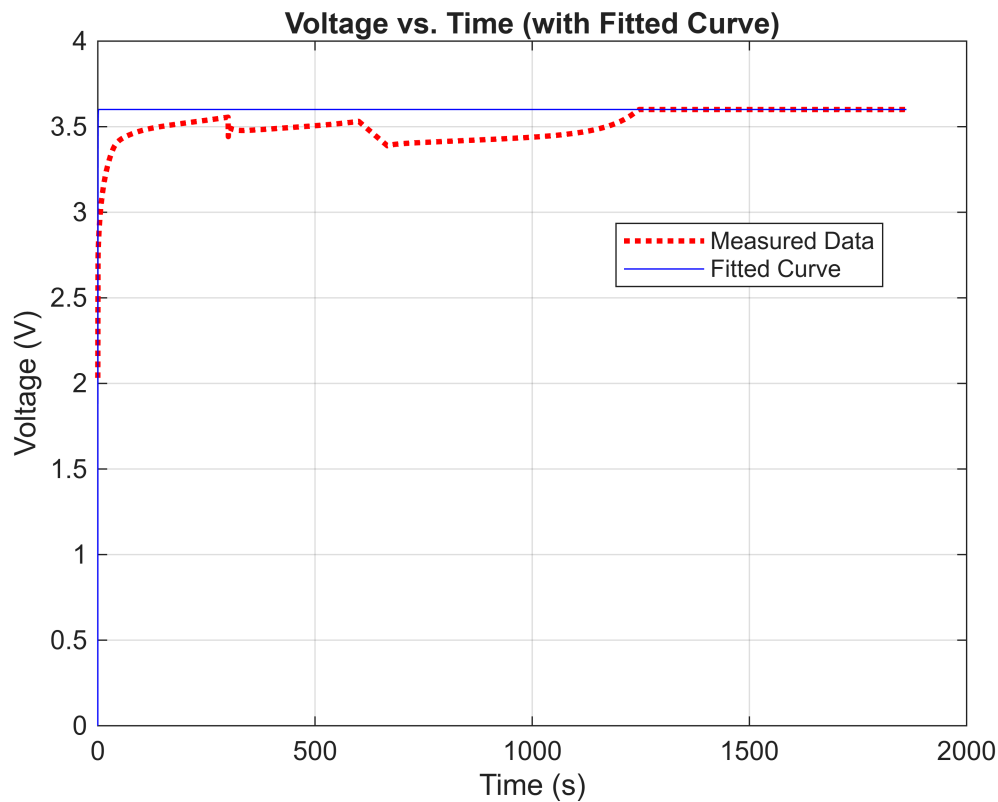
plot(time, voltage, '-r', 'LineWidth', 2.0, 'LineStyle', ':'); % Plots time
and voltage with a red line, a line width of 2.0, and a dotted line
xlabel("Time (s)"); % Labels x-axis
ylabel("Voltage (V)"); % Labels y-axis
title("Voltage vs. Time (with Fitted Curve)"); % Titles graph

hold on; % Enables hold to plot multiple curves on the same graph

plot(time, fittedCurve, '-b'); % Plots time and the fitted curve with a blue
line

legend("Measured Data", "Fitted Curve", "Location", "best"); % Enables legend
grid on; % Turns on grid

```

```
% STEP 5: Display the goodness-of-fit statistics.
```

```
tau = f.tau; % Stores tau as estimated by the fitted curve
```

```
fprintf("Tau = %.4f seconds \n", tau) % Prints tau value
```

```
Tau = 0.2279 seconds
```

```
fprintf("SSE = %.4f \n", gof.sse) % Prints SSE (Sum of Squared Errors) value
```

```
SSE = 45.7022
```

```
fprintf("R-Squared = %.4f \n", gof.rsquare) % Prints R-squared value
```

```
R-Squared = 0.2282
```

```
fprintf("DFE = %d \n", gof.dfe) % Prints DFE (Degrees of Freedom for Error) value
```

```
DFE = 611
```

```
fprintf("Adjusted R-Squared = %.4f \n", gof.adjrsquare) % Prints adjusted R-squared value
```

```
Adjusted R-Squared = 0.2282
```

```
fprintf("RMSE = %.4f \n", gof.rmse) % Prints RMSE (Root Mean Square Error) value
```

RMSE = 0.2735

Task 2: Plot the electrical terms that express how energy moves in a circuit

To visualize the battery's charging policy, write code to visualize the current, voltage, and power measurements for one charging cycle. Present the visualization as one figure with three subplots, one each for:

- Voltage vs. time
- Current vs. time
- Power vs. time

```
% STEP 1: Create a figure handle and initialize the first subplot.  
% Plot voltage vs. time.  
  
figure; % Opens figure  
subplot(3,1,1); % Divides figure into 3 subplots and selects the first  
  
plot(time, voltage, '-b', 'LineWidth', 1.5); % Plots time and voltage with a  
blue line of line width 1.5  
xlabel("Time (s)"); % Labels x-axis  
ylabel("Voltage (V)"); % Labels y-axis  
  
title("Voltage vs. Time"); % Titles graph  
  
% Optional: plot the points of notable transitions in charging dynamics in  
% the voltage vs. time plot generated above  
% Hint: explore using the gradient function  
  
dVdt = gradient(voltage, time); % Calculates rates of voltage change with  
respect to time  
  
slopeTolerance = 0.001; % Sets a slope tolerance of 0.001 to detect abnormal  
changes in slope (indicating notable events)  
  
notablePoints = find(abs(dVdt) > slopeTolerance); % Finds points where  
absolute value of slope is higher than the slope tolerance  
  
hold on; % Enables hold so multiple curves can be plotted on the same graph  
  
plot(time(notablePoints), voltage(notablePoints), 'ro', 'MarkerSize', 3) %  
Plots notable points on the time and voltage curve with red circles of  
marker size 3  
  
% STEP 2: Use a second subplot to plot current vs. time.  
  
current = cycleData.Current; % Assigns current variable to the appropriate  
cycleData column
```

```

subplot(3,1,2); % Selects the second subplot

plot(time, current, '-r', 'LineWidth', 1.5); % Plots time and current with a
red line of width 1.5
xlabel("Time (s)"); % Labels x-axis
ylabel("Current (A)"); % Labels y-axis

title("Current vs. Time"); % Titles graph

% STEP 3: Use a third subplot to plot power vs time.
% Hint:  $P = I * V$  where I is current in amps. You will first
% need to compute power before you can create this plot.

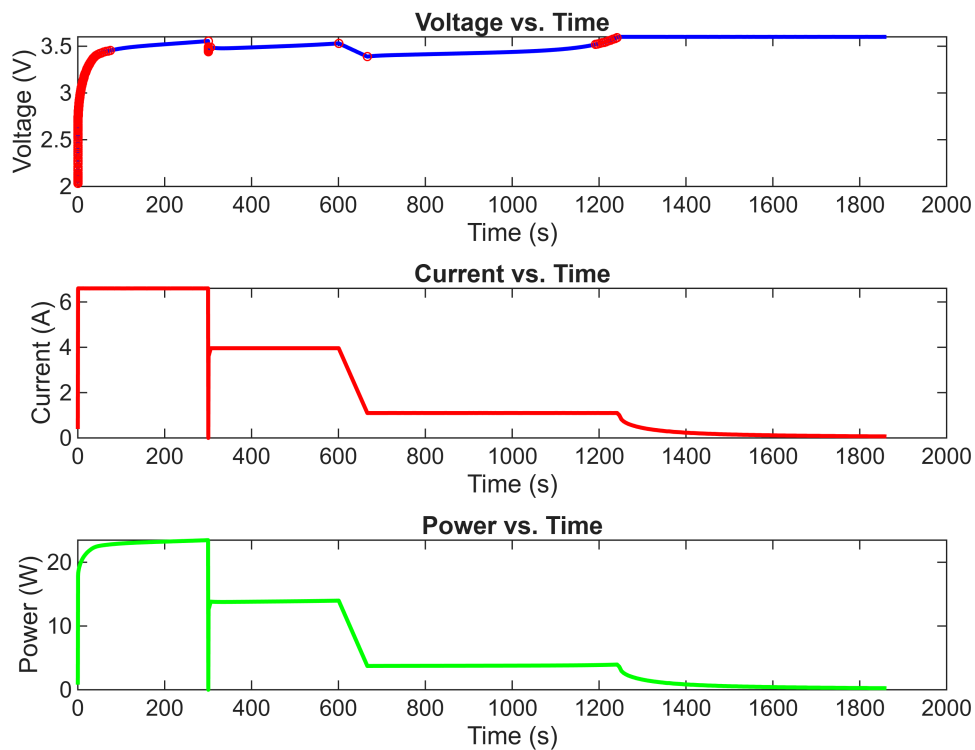
power = current .* voltage; % Calculates power by multiplying current and
voltage

subplot(3,1,3); % Selects the third subplot

plot(time, power, '-g', 'LineWidth', 1.5); % Plots time and power with a
green line of width 1.5
xlabel("Time (s)"); % Labels x-axis
ylabel("Power (W)"); % Labels y-axis

title("Power vs. Time"); % Titles graph

```



Task 3: Compute analytical results

To get started on using MATLAB for numerical integration and differentiation, start with this [doc page](#).

- Conduct a rate-of-change analysis by computing the derivative at key intervals in the voltage data
- Compute the time required to reach 80% and 100% charge
- Compute the total energy delivered to the battery by using integration to calculate the area under the power-time curve.
- Estimate the resistive energy loss
- Save the results to a summary table.

```
% STEP 1: Compute the rate of voltage change at different key time
% intervals. This helps you mathematically characterize the battery's
% charging profile and is more reliable than simply eyeballing the voltage
% vs. time plot.

% Creates key intervals based on changes in voltage vs. time graph

interval1 = time >= 0 & time <= 100;
interval2 = time > 100 & time <= 300;
interval3 = time > 300 & time <= 600;
interval4 = time > 600 & time <= 1250;
interval5 = time > 1250 & time <= 1800;

% 0-100 s
time1 = time(interval1); % Selects time values from first interval
voltage1 = voltage(interval1); % Selects voltage values from interval
dVdt1 = diff(voltage1) ./ diff(time1); % Finds dV/dt by dividing the
derivative of voltage by the derivative of time over the interval
avg1 = mean(dVdt1); % Finds the average of all dV/dt over the interval

% 100-300 s
time2 = time(interval2); % Selects time values from second interval
voltage2 = voltage(interval2); % Selects voltage values from second interval
dVdt2 = diff(voltage2) ./ diff(time2); % Finds dV/dt by dividing the
derivative of voltage by the derivative of time over the interval
avg2 = mean(dVdt2); % Finds the average of all dV/dt over the interval

% 300-600 s
time3 = time(interval3); % Selects time values from third interval
voltage3 = voltage(interval3); % Selects voltage values from third interval
dVdt3 = diff(voltage3) ./ diff(time3); % Finds dV/dt by dividing the
derivative of voltage by the derivative of time over the interval
avg3 = mean(dVdt3); % Finds the average of all dV/dt over the interval

% 600-1250 s
time4 = time(interval4); % Selects time values from fourth interval
voltage4 = voltage(interval4); % Selects voltage values from fourth interval
```

```

dVdt4 = diff(voltage4) ./ diff(time4); % Finds dV/dt by dividing the
derivative of voltage by the derivative of time over the interval
avg4 = mean(dVdt4); % Finds the average of all dV/dt over the interval

% 1250-1800 s
time5 = time(interval5); % Selects time values from fifth interval
voltage5 = voltage(interval5); % Selects voltage values from fifth interval
dVdt5 = diff(voltage5) ./ diff(time5); % Finds dV/dt by dividing the
derivative of voltage by the derivative of time over the interval
avg5 = mean(dVdt5); % Finds the average of all dV/dt over the interval

fprintf("Average voltage change from 0-100 seconds: %.4f V/s \n", avg1) %
Prints average voltage change over first interval

```

Average voltage change from 0-100 seconds: 0.3856 V/s

```

fprintf("Average voltage change from 100-300 seconds: %.4f V/s \n", avg2) %
Prints average voltage change over second interval

```

Average voltage change from 100-300 seconds: 0.0003 V/s

```

fprintf("Average voltage change from 300-600 seconds: %.4f V/s \n", avg3) %
Prints average voltage change over third interval

```

Average voltage change from 300-600 seconds: -0.0107 V/s

```

fprintf("Average voltage change from 600-1250 seconds: %.4f V/s \n", avg4)
% Prints average voltage change over fourth interval

```

Average voltage change from 600-1250 seconds: 0.0002 V/s

```

fprintf("Average voltage change from 1250-1800 seconds: %.4f V/s \n", avg5)
% Prints average voltage change over fifth interval

```

Average voltage change from 1250-1800 seconds: -0.0000 V/s

```

% STEP 2: Compute the time required to reach 80% charge and 100% charge

```

```

v80 = 0.80 * max(voltage); % Calculates 80% of maximum voltage
indexV80 = find(voltage >= v80, 1); % Finds the index number of the first
point that reaches a voltage of v80
time80 = time(indexV80); % Finds the first time where v80 is reached

```

```

% Time to Reach 100% Charge

```

```

[maxVoltage, indexMaxVoltage] = max(voltage); % Finds the maximum voltage
and its index number
time100 = time(indexMaxVoltage); % Finds the time where max voltage first
occurs
fprintf("Time to reach 80 Percent Charge: %.3f seconds \n", time80) % Prints
time to reach 80% charge

```

Time to reach 80 Percent Charge: 2.682 seconds

```
fprintf("Time to reach 100 Percent Charge: %.3f seconds \n", time100) %  
Prints time to reach 100% charge
```

Time to reach 100 Percent Charge: 1246.495 seconds

```
% STEP 3: Use integration to calculate the area under the power-time curve.  
% This represents the total energy delivered to the battery during charging.  
% Hint: explore using the trapz function
```

```
totalEnergy = trapz(time, power); % Calculates total energy delivered (in  
Joules) by integrating the power vs. time curve  
totalEnergykJ = totalEnergy/1000; % Calculates total energy in kJ
```

```
fprintf("Total Energy Delivered to the Battery: %.3f kJ \n", totalEnergykJ)  
% Prints total energy
```

Total Energy Delivered to the Battery: 14.222 kJ

```
% STEP 4: Estimate the resistive energy loss.  
% Hint: use Joule's heating formula ( $P = I^2 * R$ ) to estimate power loss in  
% watts. The energy loss is given as  $E = P * t$ , where  $t$  is time in seconds  
% and energy is in joules.
```

```
resistance = cycleData.Internal_Resistance; % Extracts internal resistance  
from dataset
```

```
powerLoss = current.^2 .* resistance; % Calculates power using Joule's  
heating formula  
avgPowerLoss = mean(powerLoss); % Finds average power loss over every point  
in the dataset
```

```
allTime = time(end) - time(1); % Extracts all time values  
energyLoss = avgPowerLoss * allTime; % Calculates energy loss by multiplying  
power and time
```

```
fprintf("Average Power Loss: %.3f Watts \n", avgPowerLoss) % Prints average  
power loss in Watts
```

Average Power Loss: 0.243 Watts

```
fprintf("Resistive Energy Loss: %.3f Joules \n", energyLoss) % Prints  
resistive energy loss in Joules
```

Resistive Energy Loss: 452.379 Joules

```
format short g; % Formats all following numbers with 5 significant figures
```

```
table(Vmax, tau, time80, time100, totalEnergykJ, avgPowerLoss, energyLoss,
...
'VariableNames', {'MaximumVoltage(V)', 'Tau(s)', ...
'TimeFor80PercentCharge(s)', 'TimeForFullCharge(s)', ...
'TotalEnergyDelivered(kJ)', 'AveragePowerLoss(W)', ...
'ResistiveEnergyLoss(J)'})) % Creates table containing the named
variables and assigns appropriate column names for readability
```

ans = 1×7 table

	MaximumVoltage(V)	Tau(s)	TimeFor80PercentCharge(s)	TimeForFullCharge(s)
1	3.6004	0.2279	2.6823	1246.5

Interpretation of Results

Summarize your findings by interpreting their physical/engineering meaning (for example, why would engineers want to compute the analytical results in Task 3?):

- Our results are highly practical and relevant in engineering. This project effectively demonstrates the charging cycle of a lithium-ion battery, which is widely used in commonplace objects like cell phones and electric cars. The analytical results displayed above are crucial in modeling characteristics such as charge, lifespan, safety, and efficiency of a lithium-ion battery. Computing charging times would benefit engineers striving to understand and maximize the lifespan of an electric car battery. Computing resistive energy loss would help engineers cut unnecessary losses and assess the efficiency of their systems. The tau value estimates how quickly the battery charges, information that may be used to improve user experience and overall charging times. These physical applications, among others, are demonstrated through our code.

What are some limitations of your work?

- Our work is limited by the dataset provided and assumes that the voltage equation is an accurate model for lithium-ion battery charging. Our results are based on this singular dataset and thus may not adequately illustrate analytical results for all lithium-ion batteries, especially those of voltages higher than the 3.6 V provided. Some values calculated in this project, such as tau and resistive energy loss, are estimates derived using external formulas and are not expressly provided in the dataset. This dataset is assumed to be ideal and does not take into account several real-world factors such as resistor tolerance, circuit construction, temperature fluctuations, or miscellaneous error that may affect experimental results.

What are practical next steps?

- Practical next steps would be to repeat this experiment under different conditions to evaluate the validity of the data provided. Additional quantities such as battery capacity and temperature may be factored into future trials in order to account for a broader spectrum of possibilities, creating a more comprehensive analysis of lithium-ion battery charging and improving the accuracy of estimates such as tau and energy loss values. Once the charging model has been perfected, engineers may use detailed datasets to improve battery life, refine charging cycles, and build towards future technological advancements.

Optional Extension (intermediate):

The lithium-ion battery from the example dataset follows a Constant Current-Constant Voltage (CC-CV) charging profile (standard for lithium ion batteries), where constant current is initially applied to rapidly increase voltage and then switches to constant voltage to safely complete the charge as current decreases.

As an optional extension of this project:

Identifying Charging Phases

- **Objective:** Utilize the data from one cycle. Identify the charging phases in the dataset and split the data into two regions: CC region (where voltage is increasing and current is approximately constant) and CV region (constant voltage, decreasing current). Clearly mark the transition points on plots. Optionally, confirm using rate of voltage change analysis (the derivative should be 0 at the start of the CV phase).

```
% STEP 1: Utilize the data from one cycle. Identify the charging phases in
the dataset and split the data into
% two regions: CC region (where voltage is increasing and current is
approximately constant) and CV region
% (constant voltage, decreasing current). Clearly mark the transition points
on plots.
% Optionally, confirm using rate of voltage change analysis (the derivative
should be 0 at the start of the CV phase).

dVdt = gradient(voltage, time); % Calculates the derivative of voltage with
respect to time (slope of graph)

for i = 1:length(dVdt) % Creates variable i that runs for the length of dVdt
for counter

    if all(abs(dVdt(i:i+99)) < 0.0001) % Finds 100 consecutive dVdt values
that are essentially 0

        transitionIndex = i; % Sets transition index to the first point found

        break % Breaks loop

    end % Ends if loop

end % Ends for loop

transitionTime = time(transitionIndex); % Finds corresponding time for
transition point

fprintf("Time where CV region begins: %.2f seconds \n", transitionTime); %
Prints time of transition between CC and CV phases
```

```
Time where CV region begins: 1251.50 seconds
```



```
% Voltage vs. Time Plot
```

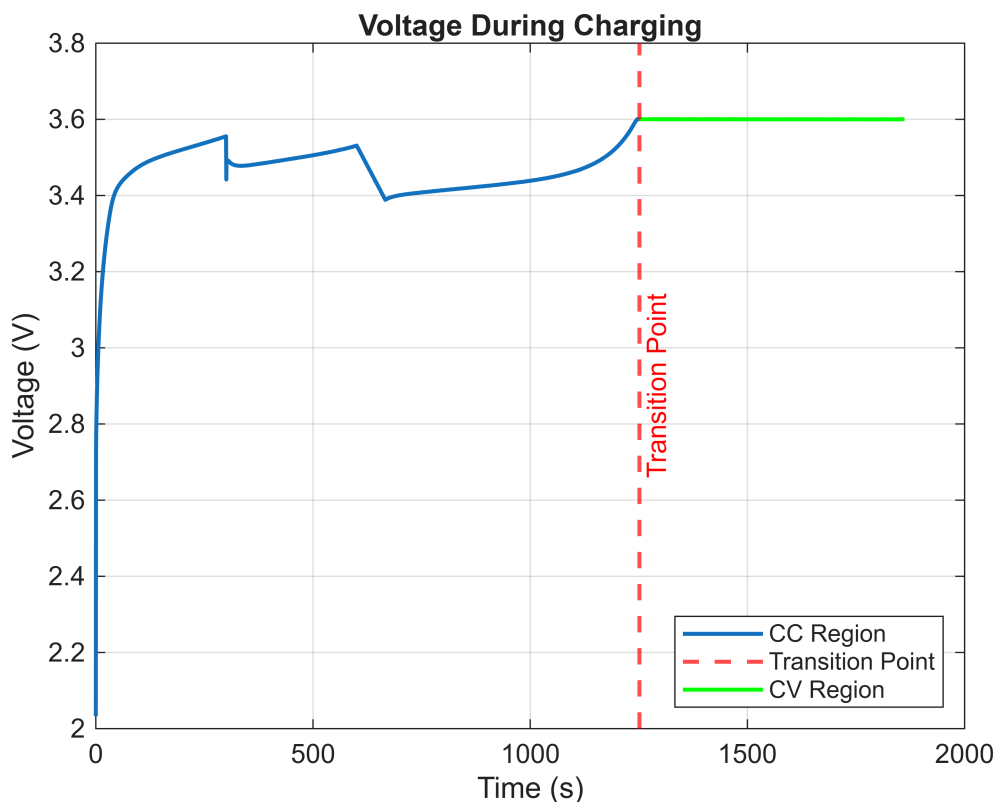
```
figure; % Opens figure
```

```
plot(time(1:transitionIndex), voltage(1:transitionIndex), 'LineWidth', 1.5);  
% Plots time and voltage with line width 1.5 before transition point  
xlabel('Time (s)'); % Labels x-axis  
ylabel('Voltage (V)'); % Labels y-axis  
xline(transitionTime, 'r--', 'Transition Point', 'LineWidth', 1.5,  
'LabelVerticalAlignment', 'middle');  
% Creates dotted, red line at the transition point with line width 1.5  
and aligns text accordingly  
title('Voltage During Charging'); % Titles graph
```

```
hold on; % Enables hold to plot multiple curves on the same graph
```

```
plot(time(transitionIndex:end), voltage(transitionIndex:end), '-g',  
'LineWidth', 1.5); % Plots green line after transition point
```

```
legend('CC Region', 'Transition Point', 'CV Region', 'Location',  
'southeast') % Creates appropriate legend  
grid on; % Turns on grid
```



```
% Current vs Time Plot
```

```
% Code generated using the same procedures as above
```

```
figure;
```

```
plot(time(1:transitionIndex), current(1:transitionIndex), 'LineWidth', 1.5);  
% Plots time and current with line width 1.5 before transition point
```

```
xlabel('Time (s)')
```

```
ylabel('Current (A)')
```

```
xline(transitionTime, 'r--', 'Transition Point', 'LineWidth', 1.5,  
'LabelVerticalAlignment', 'middle');
```

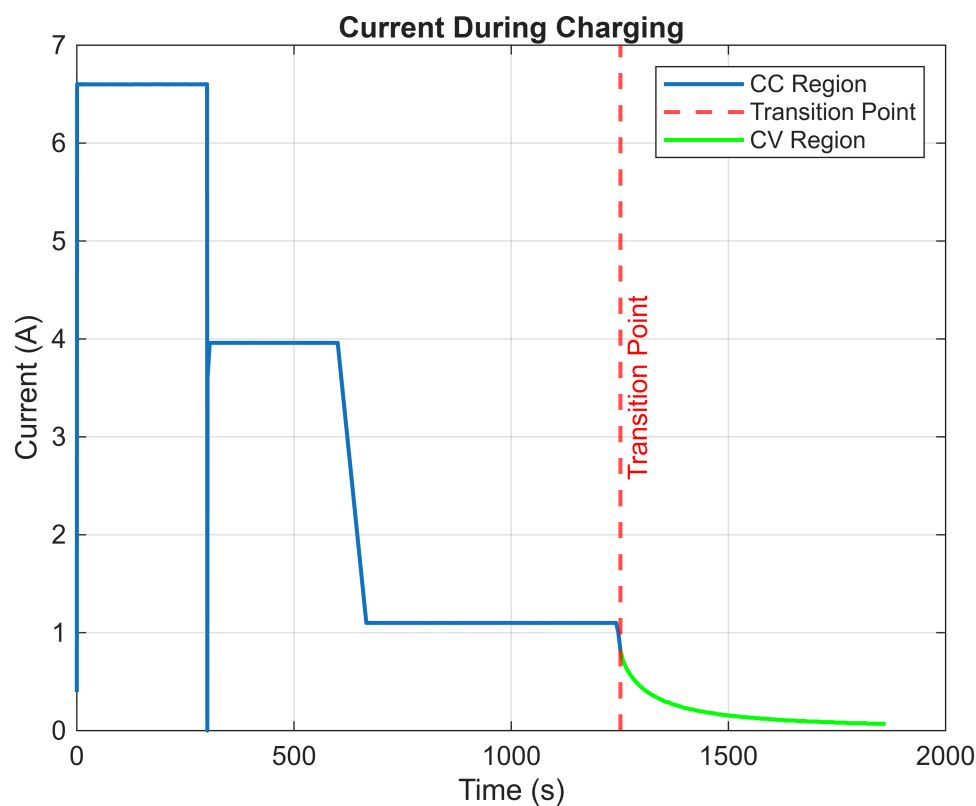
```
title('Current During Charging')
```

```
hold on;
```

```
plot(time(transitionIndex:end), current(transitionIndex:end), '-g',  
'LineWidth', 1.5); % Plots green line after transition point
```

```
legend('CC Region', 'Transition Point', 'CV Region', 'Location', 'northeast')
```

```
grid on;
```



Analyzing Each Phase Separately

- **Objective:** Analyze each phase separately. Fit the same exponential model as above to the CC phase and fit an exponential decay model to the plot of current vs. time during the CV phase.

```
% STEP 2: Analyze each phase separately. Fit the same exponential model as
above to the CC phase and fit
% an exponential decay model to the plot of current vs. time during the CV
phase.

ccData = cycleData(cycleData.DateTime <= transitionTime, :); % Assigns data
before (and at) the transition time to the CC region
cvData = cycleData(cycleData.DateTime > transitionTime, :); % Assigns data
after the transition time to the CV region

% CC voltage model ( $V(t) = V_{max}(1 - e^{-t/\tau})$ )

Vmax_CC = max(ccData.Voltage); % Finds maximum voltage in the CC region

[f, gof] = fit(ccData.DateTime, ccData.Voltage, ft, 'problem', Vmax_CC); %
Returns the fitted curve and goodness-of-fit statistics according to the
named parameters

figure; % Opens figure

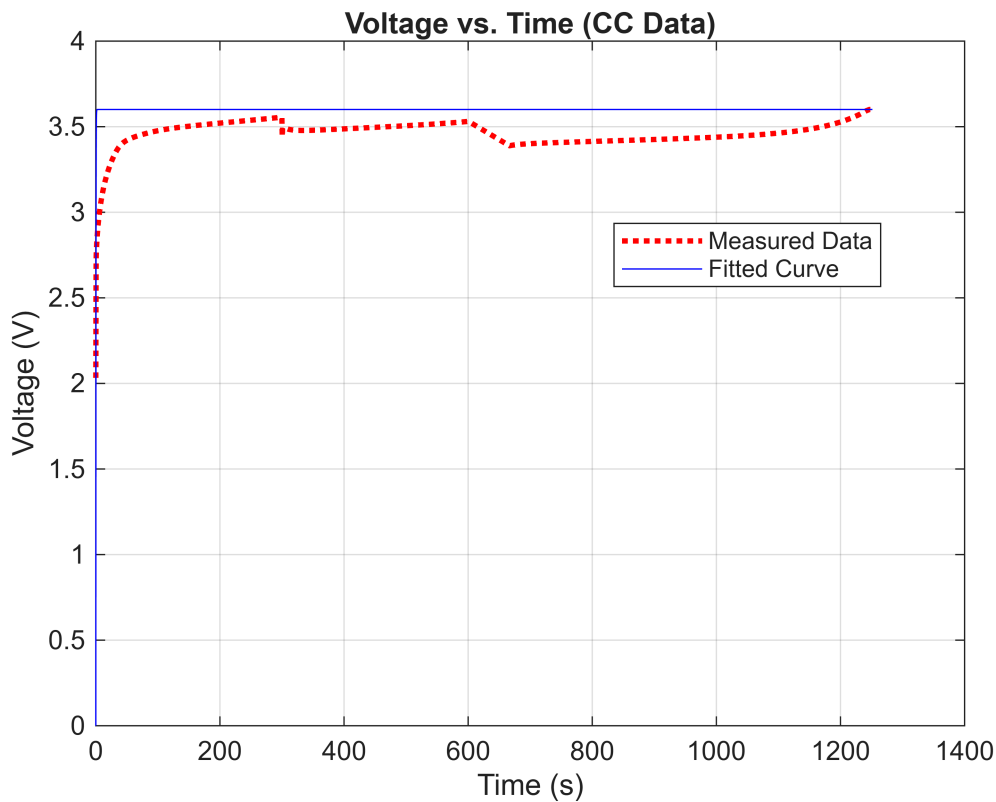
plot(ccData.DateTime, ccData.Voltage, '-r', 'LineWidth', 2.0, 'LineStyle',
':'); % Plots CC region time and voltage with a dotted red line of width 2.0
xlabel("Time (s)"); % Labels x-axis
ylabel("Voltage (V)"); % Labels y-axis
title("Voltage vs. Time (CC Data)"); % Titles graph

hold on; % Enables hold to plot multiple curves on the same graph

fittedCurve2 = f(ccData.DateTime); % Applies the fitted curve to each time
value in CC region

plot(ccData.DateTime, fittedCurve2, '-b'); % Plots CC region time and the
fitted curve with a blue line

legend("Measured Data", "Fitted Curve", "Location", "best"); % Creates
appropriate legend
grid on; % Turns on grid
```



```
% CV Current Model ( $I(t) = Ae^{(-t/\tau)}$ )
```

```
cvTime = cvData.DateTime - cvData.DateTime(1); % Resets time at 0 for the CV  
region for simplicity
```

```
A_current = max(cvData.Current); % Assigns the coefficient A the maximum  
current in CV region
```

```
tauCurrent = 50; % Sets arbitrary starting point for tau
```

```
foCurrent = fitoptions('Method', 'NonlinearLeastSquares', ...  
    'StartPoint', [tauCurrent A_current]); % Selects fit options with a  
nonlinear least squares regression curve and begins at the starting points
```

```
ftCurrent = fittype('A * exp(-time/tau)', ...  
    'independent', 'time', ...  
    'coefficients', {'tau', 'A'}, ...  
    'options', foCurrent); % Defines the current equation for the curve fit,  
time as the independent variable,  
% tau and A as unknown coefficients, and uses  
the fit options above
```

```
[f, gof] = fit(cvTime, cvData.Current, ftCurrent); % Returns the fitted  
curve and goodness-of-fit statistics according to the named parameters
```

```

figure; % Opens figure

plot(cvTime, cvData.Current, '-r', 'LineWidth', 2.0, 'LineStyle', ':'); %
Plots CV region time and current with a dotted red line of width 2.0
xlabel("Time (s)"); % Labels x-axis
ylabel("Current (A)"); % Labels y-axis
title("Current vs. Time (CV Data)"); % Titles graph

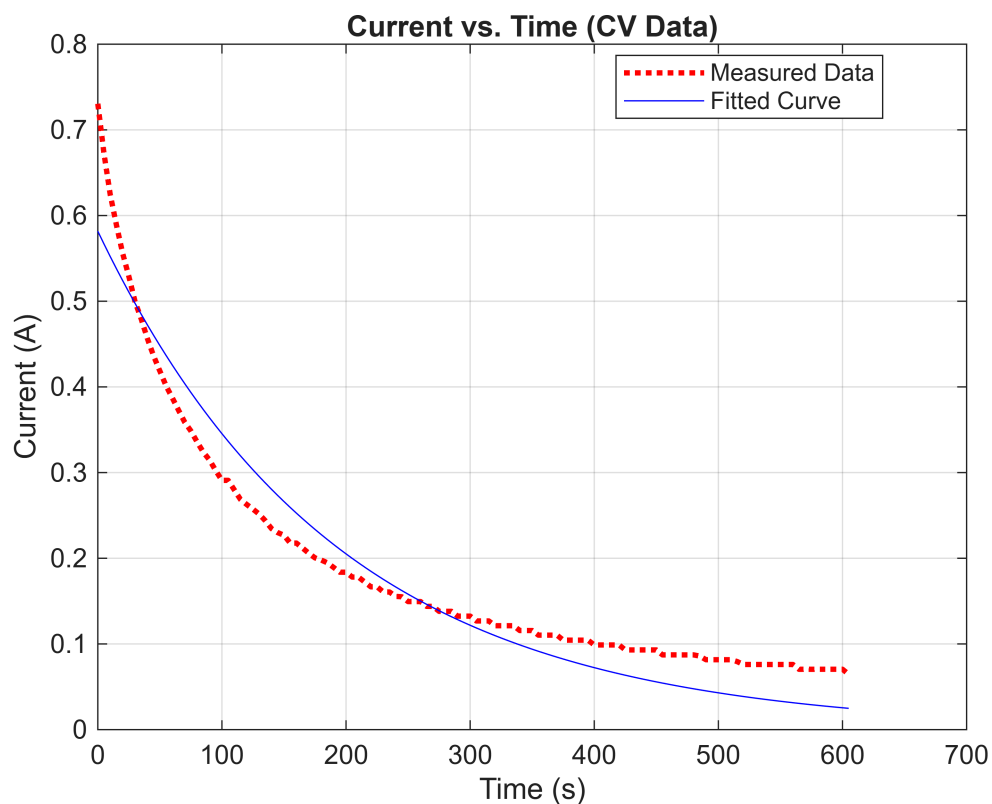
hold on; % Enables hold to plot multiple curves on the same graph

fittedCurve3 = f(cvTime); % Applies fitted curve to each time value in the
CV region

plot(cvTime, fittedCurve3, '-b') % Plots CV region time and fitted curve
with a blue line

legend("Measured Data", "Fitted Curve", "Location", "best"); % Creates
appropriate legend
grid on; % Turns on grid

```



Comparing Power and Efficiency of Phases

- **Objective:** Compare power vs. time during each phase and compute total energy delivered during each phase. Identify when charging is most efficient and when most energy is delivered.

```
% STEP 3: Compare power vs. time during each phase and compute total energy
delivered during each phase.
% Identify when charging is most efficient and when most energy is delivered.

ccPower = ccData.Voltage .* ccData.Current; % Calculates power for CC region
using P = VI
cvPower = cvData.Voltage .* cvData.Current; % Calculates power for CV region
using P = VI

% Plot and Compare

figure; % Opens figure

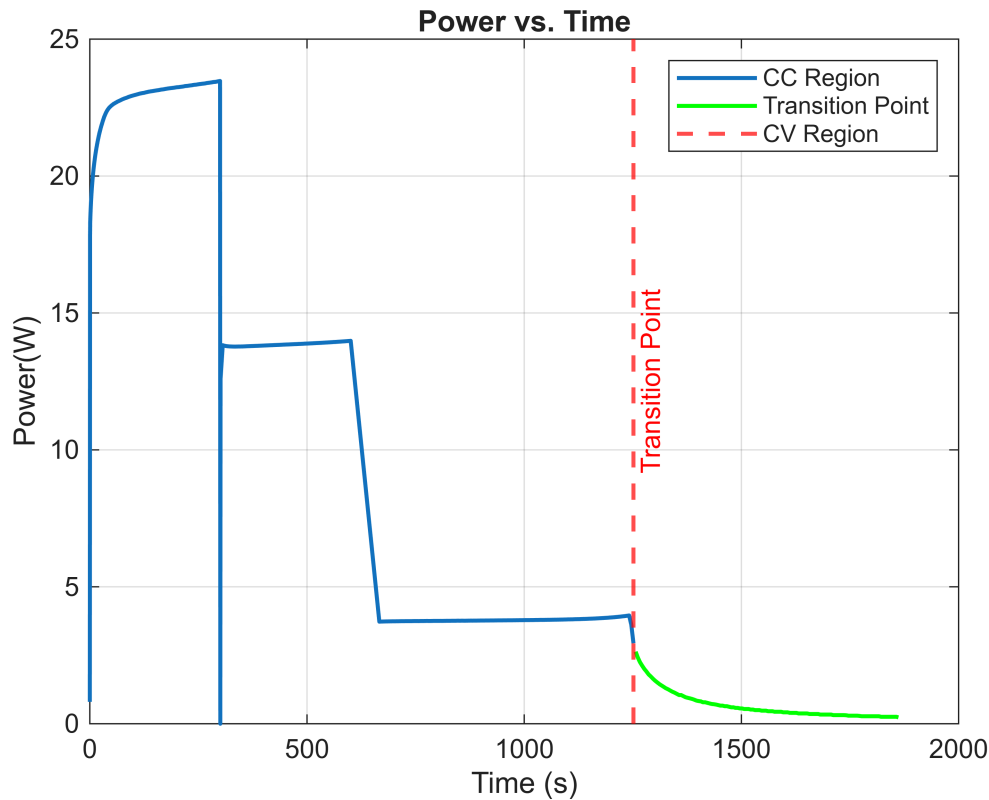
plot(ccData.DateTime, ccPower, 'LineWidth', 1.5) % Plots CC region time and
power with line width 1.5

hold on; % Enables hold to plot multiple curves on the same graph

plot(cvData.DateTime, cvPower, '-g', 'LineWidth', 1.5); % Plots CV region
time and power with green line of width 1.5
xlabel('Time (s)') % Labels x-axis
ylabel('Power(W)') % Labels y-axis
xline(transitionTime, 'r--', 'Transition Point', 'LineWidth', 1.5,
'LabelVerticalAlignment', 'middle');
    % Creates dotted, red line at the transition point with line width 1.5
and aligns text accordingly
title('Power vs. Time') % Titles graph

legend('CC Region', 'Transition Point', 'CV Region', 'Location',
'northeast') % Creates appropriate legend

grid on; % Turns on grid
```



```
% Compute Total Energy
```

```
ccEnergy = trapz(ccData.DateTime, ccPower); % Computes energy delivered
during CC phase by integrating the power vs. time curve
```

```
cvEnergy = trapz(cvData.DateTime, cvPower); % Computes energy delivered
during CV phase by integrating the power vs. time curve
```

```
fprintf("Energy delivered during CC phase: %.2f J \n",ccEnergy) % Prints
energy delivered during CC phase
```

```
Energy delivered during CC phase: 13807.05 J
```

```
fprintf("Energy delivered during CV phase: %.2f J \n",cvEnergy) % Prints
energy delivered during CV phase
```

```
Energy delivered during CV phase: 400.63 J
```

```
% Power Loss
```

```
ccPowerLoss = (ccData.Current).^2 .* ccData.Internal_Resistance; % Computes
power lost during CC phase with Joule's heating formula
```

```
cvPowerLoss = (cvData.Current).^2 .* cvData.Internal_Resistance; % Computes
power lost during CV phase with Joule's heating formula
```

% Energy Loss

```
ccEnergyLoss = trapz(ccData.DateTime, ccPowerLoss); % Computes energy loss
during CC phase by integrating the power loss vs. time curve
```

```
cvEnergyLoss = trapz(cvData.DateTime, cvPowerLoss); % Computes energy loss
during CV phase by integrating the power loss vs. time curve
```

% Efficiency

```
ccEfficiency = ((ccEnergy - ccEnergyLoss) / ccEnergy) * 100; % Computes
efficiency of CC region by dividing useful output by total output
```

```
cvEfficiency = ((cvEnergy - cvEnergyLoss) / cvEnergy) * 100; % Computes
efficiency of CV region by dividing useful output by total output
```

```
fprintf("CC Phase Efficiency: %.2f%% \n", ccEfficiency) % Prints CC
efficiency for comparison
```

CC Phase Efficiency: 97.57%

```
fprintf("CV Phase Efficiency: %.2f%% \n", cvEfficiency) % Prints CV
efficiency for comparison
```

CV Phase Efficiency: 99.87%

Despite the CC phase delivering more energy to the battery, the CV phase is the more efficient according to the above calculations.

Modeling and Analyzing Different Charging Profiles Using Simulink

- **Objective:** CC-CV is one type of charging profile. Use Simscape Battery to model and compare different charging profiles (such as constant current, multi-stage/smart charging, or fast electric vehicle charging). Plot the simulation results (charging/discharging current, battery voltage, and battery temperature) for different charging profiles. This [example](#) shows how to simulate a CC-CV battery charging profile in Simscape, and shows the example plots of the simulation results. You can also use the [Battery Builder app](#) to export a custom battery object to Simulink for circuit modeling and analysis.

Description of CC-CV (Constant Current - Constant Voltage) vs. MSCCCV (Multi-Stage Constant Current - Constant Voltage):

- The CC-CV charging method uses a constant current of about 1.5 A to charge the battery. As the battery charges, the voltage increases until it reaches its upper limit. The charging method then controls the current to prevent the battery voltage from becoming too high. This makes CC-CV a simple and effective charging method.

- The Multi-Stage charging method changes the charging current in steps. In the simulation, the current starts around 1.5 A, then decreases to 1 A, and finally to 0.5 A. This allows the battery to charge quickly at first and more slowly as it becomes more charged

Comparison of CC-CV vs. MSCCCV:

- The voltage follows a similar pattern for both methods. However, CC-CV reaches a slightly higher voltage, while Multi-Stage charging provides a more gradual voltage increase because the current is reduced in stages.
- There is also a small difference in battery temperature. The CC-CV method reaches about 298.4 K, while the Multi-Stage method reaches about 298.3 K. This suggests that reducing the current in stages can slightly reduce battery heating.
- Overall, CC-CV is simpler and charges the battery more aggressively, while Multi-Stage charging provides more control and produces slightly less heating. Multi-Stage charging may be better for reducing stress on the battery, although it may take longer to fully charge.

```
% STEP 4: CC-CV is one type of charging profile. Use Simscape Battery to
model and compare different charging profiles (such as constant current,
multi-stage/smart charging, or fast electric vehicle charging).
% Plot the simulation results (charging/discharging current, battery
voltage, and battery temperature) for different charging profiles.
% This example shows how to simulate a CC-CV battery charging profile in
Simscape, and shows the example plots of the simulation results.
% You can also use the Battery Builder app to export a custom battery object
to Simulink for circuit modeling and analysis.
```

```
% Part 1: Loading the Simulink Models into MATLAB
```

```
modelName_CC = 'CCCV'; % Define the name of the Simulink model that
implements the CC-CV
modelName_MS = 'MSCCCV'; % Defines the name of the Simulink model that
implements Multi-Stage Charging

warning('off', 'all'); % Disables the warning of an unnecessary issue
load_system(modelName_CC); % Loads the CC-CV Simulink model
simOut_CC = sim(modelName_CC); % Runs the CC-CV Simulink model and stores
all simulation outputs in the variable
warning('on', 'all'); % Re-enables the warnings after CC-CV simulation has
completed
CC = simOut_CC.logout; % Extracts the logged signals from the CC-CV
simulation (Includes Voltage, Temperature, and Current)

load_system(modelName_MS); % Loads the MSCCCV Simulink model (Multi-Stage
Charging)
```

```

simOut_MS = sim(modelName_MS); % Runs the MSCCCV Simulink model and stores
all simulation outputs in the variable
MS = simOut_MS.logout; % Extracts the logged signals from the CC-CV
simulation (Includes Voltage, Temperature, and Current)

% Part 2: Plotting the CC-CV Graphs
% (Constant Current - Constant Voltage)

ylabel = {"Voltage (V)", "Temperature (K)", "Current (A)"}; % Labels for
y-axes of the graphs

figure % Opens figure
subplot(3,1,1); % Creates a 3x1 graph of subplots and selects the first

for i = 1:CC.numElements % Iterates through the elements (Voltage, Temp, and
Current) to create graphs
    signal = CC.getElement(i); % Retrieves the values of the i-th signal

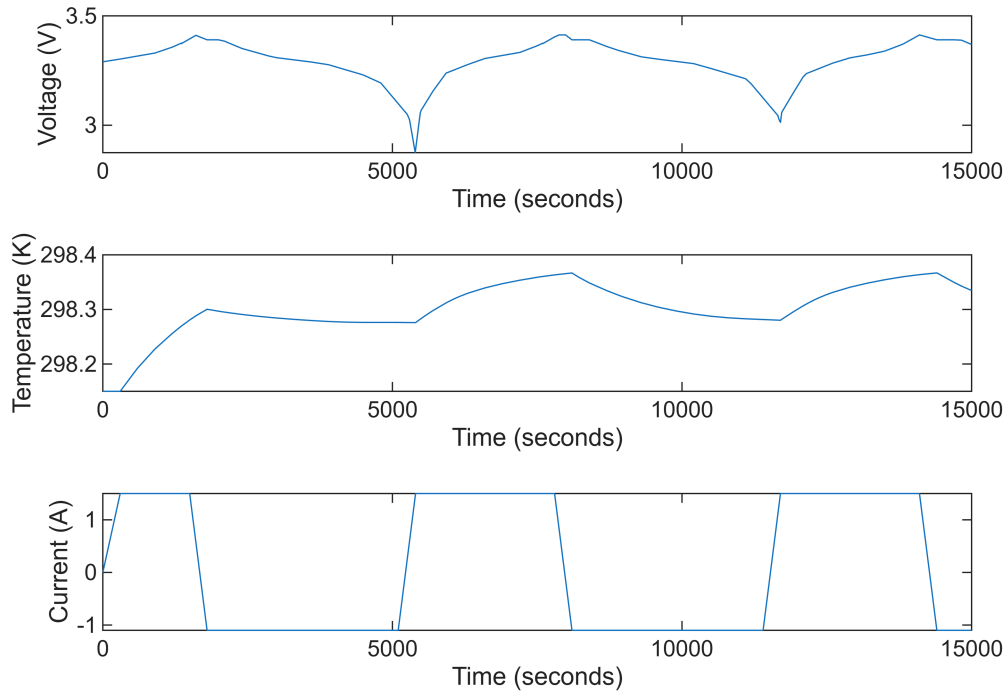
    subplot(3,1,i) % Selects the next available subplot
    plot(signal.Values.Time, signal.Values.Data); % Plots the Data Value vs.
Time

    ylabel(ylabel{i}); % Labels the y-axis according to the current signal
being plotted
    xlabel("Time (seconds)"); % Labels the x-axis
end % Ends for loop

sgtitle("CC-CV Battery Charging Algorithm"); % Creates overall title above
all three CC-CV plots

```

CC-CV Battery Charging Algorithm



```
% Part 3: Plotting the MSCCV Graphs
% (Multi-Stage Constant Current - Constant Voltage)

figure % Opens figure

subplot(3,1,1) % Creates a 3x1 graph of subplots and selects the first

for i = 1:MS.numElements % Iterates through the elements (Voltage, Temp, and
Current) to create graphs
    signal = MS.getElement(i); % Retrieves the values of the i-th signal

    subplot(3,1,i) % Selects the next available subplot
    plot(signal.Values.Time, signal.Values.Data); % Plots the Data Value vs.
Time

    ylabel(ylabels{i}); % Labels the y-axis according to the current signal
being plotted
    xlabel("Time (seconds)"); % Labels the x-axis
end % Ends for loop

sgtitle("Multi-Stage Battery Charging Algorithm"); % Creates overall title
above all three CC-CV plots
```

Multi-Stage Battery Charging Algorithm

